

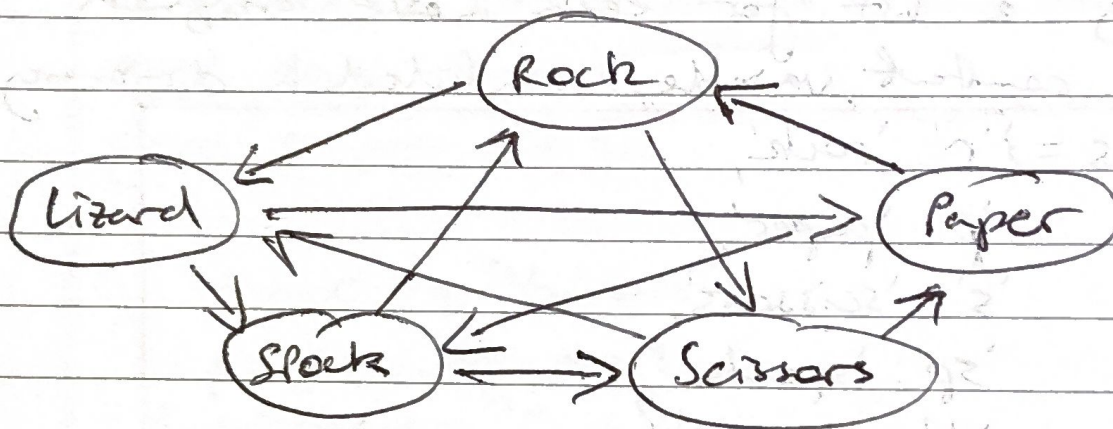
#28 - Assignment - RPS - Bonus - Features

16/06/2025

Bonus Features:

1. Lizard - Spock

A variation of Rock Paper Scissors, by adding two more options - Lizard and Spock.



IF ((player == "rock", computer == "scissors") or
(player == "rock", computer == "lizard") or
(player == "paper", computer == "rock") or
(player == "paper", computer == "lizard") or
(player == "scissors", computer == "paper") or
(player == "scissors", computer == "lizard") or
(player == "spock", computer == "scissors") or
(player == "spock", computer == "rock") or
(player == "lizard", computer == "spock") or
(player == "lizard", computer == "paper")):

Implementing the above into the program.

18/06/2025

(2) Shortened Input:

Typing the full word "rock", "paper" etc is tiring. Update the program so the user can type "r" for "rock" or "p" for "paper" etc. Note there are five options starting with "s", "scissors" and "spock". How do you resolve that?

- Rather than having a list for each move assigned to `VALID_CHOICES` constant, I implemented a dictionary

```
VALID_CHOICES = {'r': 'rock',  
                 'p': 'paper',  
                 's': 'scissors',  
                 'sp': 'spock',  
                 'l': 'lizard'}
```

- The prompt for the user to choose a move, as follows.

```
prompt("Choose one of the following moves:")  
for (key, value) in VALID_CHOICES.items():  
    prompt(f"{key} for {value}")  
user_choice = input()
```

- For the computer's choice:

```
computer_choice = random.choice(list(VALID_CHOICES.keys()))
```

(3) Best of Five

Keep tally of the player and computer wins.
First to 3 wins the match.

I implemented three functions:

- one; to update the score
- two; to determine if there is a match winner at the end of each game

score declared as global variable. so used global in function def.

13/06/2025

- three; a function to end the match once a player/computer had reached 3 wins.

code omitted.

def match_points(winner):

"""Updates the score at the end of each game"""

global player_score

global computer_score

if winner == "You win":

player_score += 1

elif winner == "Computer wins":

computer_score += 1

def match_winner(player_score, computer_score):

"""Determines if there is a match winner after each game."""

if player_score == 3:

prompt("PLAYER WINS THE MATCH!")

elif computer_score == 3:

prompt("Computer wins the match!")

def end_match(player_score, computer_score):

"""Determines if the game can be ended based on score."""

if player_score == 3 or computer_score == 3

return True

code omitted

while True

player_score = 0

computer_score = 0

initialization of variables.

19/06/2025

while True

```
    prompt("Choose one of the following moves:")  
    for (key, value) in VALID_CHOICES.items():  
        prompt(f"'{key}' for '{value}'")  
    user_choice = input()
```

```
while user_choice not in VALID_CHOICES.keys():  
    prompt("You entered an invalid move. Try again.")  
    user_choice = input()
```

```
computer_choice = random.choice(list(VALID_CHOICES.keys()))  
prompt(f"You chose {user_choice}, the computer chose {computer_choice}.")
```

```
prompt(display_winner(user_choice, computer_choice))  
match_points(display_winner(user_choice, computer_choice))  
prompt(f"The score is player {player_score} : {computer_score}  
computer")
```

```
match_winner(player_score, computer_score)
```

```
if end_match(player_score, computer_score):  
    break
```

```
answer = ""
```

```
while not (answer.startswith('y') or answer.startswith('n')):
```

```
    prompt("Would you like to play another match? (y/n)")
```

```
    answer = input().lower()
```

```
    if not (answer.startswith('y') or answer.startswith('n')):
```

```
        prompt("Provide a valid response.")
```

```
if answer[0] == 'n':  
    break
```


PLYLINT

19/06/2025

After running Pylint, I became aware that using global statements within a function is not good practice.

20/06/2025

The reason global statement is required when using a global variable is because you are performing a reassignment when augmenting it. Considering this, rather than keeping score with a immutable object, better to use a mutable object: list or dictionary.

i.e.,

player_score = [0]
computer_score = [0] } ← (list)

or

score = {'player': 0, 'computer': 0} ← (dictionary)

Using a list is not idiomatic considering it only contains one element. More of a hack-a-round.

Best to use dictionary for now and ideally classes later on when that is learnt.

LSBOT

After submitting my code to LSBOT for review two - condensed the if conditionals into a dictionary of winning moves.

e.g., {'rock': ['scissors', 'lizard']}

⋮

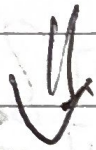
}

each winning move can beat two losing moves.

21/06/2025

- rather than invoking `display_winner()` inside the `match_points()` function invocation, assigned `display_winner()` to a variable and then invoked that variable as an argument for `match_winner`.

old `prompt(display_winner(user_choice, computer_choice))`
`match_points(display_winner(user_choice, computer_choice),`
`match_score)`



new

`result = display_winner(user_choice, computer_choice)`
`prompt(result)`

`match_points(result, match_score)`

- also included greater flexibility in input validation, as long as startswith of the input corresponded to a valid option.